Weather Company Data | Data Visualization - Feature Data Service | Common Usage Guide

Domain Portfolio: Data Visualization | Domain: Common | API Name: Feature Data Service - Common Usage Guide

Overview

The **Feature Data Service** (FDS) API provides geographic features for a number of products. The client can use these features to render a visual representation of data.

Each **product** provides homogeneous data, typically from a single source, and the visualizations resulting from each product is generally called a "layer." Each product has a unique **product key** which is invoked as a path parameter in calls to the API, and access to each product's data is granted to client-specific **API keys**. Clients use the API key in every call to the API, invoked as a required query parameter.

A **feature** can be a point or set of points, a polygon or set of polygons, a linestring or set of linestrings, or any other valid GeoJSON type. Each feature is described by a set of geographic coordinates and a set of properties. Each feature is uniquely identified by the *combination* of its product key, feature key, and valid time. The **valid time** is used to assign the feature to a unique feature set.

Clients must use the following multi-step workflow to use the API.

1. **Get Product Info** - Retrieve current information for a specific product, including time-based labels for the feature sets that are currently available.
2. **Get Coverage Info** *(Optional)* - Retrieve information on which tiles contain features, for a single feature set within a specific product.
3. **Get Features for a Single Tile** - Retrieve all geographic features for a single tile, taken from a single feature set within a specific product.
4. **Get Additional Properties for a Single Feature** *(Optional)* - Retrieve additional details for a single feature, uniquely identified by its product key, feature ID, and valid time.

HTTP Headers and Data Lifetime - Caching and Expiration

For details on appropriate header values as well as caching and expiration definitions, please see **The Weather Company Data | API Common Usage Guide**.

URL Construction

Step 1: Get Product Info
Required Parameters: **productKey,** apiKey=**yourApiKey** || **Optional Parameters:** meta, max-times https://api.weather.com/v2/vector-api/products/**<productKey>**/info?**apiKey=yourApiKey**
The [**/products/{productKey}/info**] request provides the labels for the feature sets that are currently available. These labels are required as input for the subsequent [**/products/{productKey}/features**] request, and they are invoked in that request's **'time'** parameter.
https://api.weather.com/v2/vector-api/products/901/info?meta=true&max-times=12&apiKey=yourApiKey
Step 2: Get Coverage Info *(Optional)*
Required Parameters: **productKey, time,** apiKey=**yourApiKey**

https://api.weather.com/v2/vector-api/products/**<productKey>**/coverage?time=**<time>**&**apiKey=yourApiKey**
https://api.weather.com/v2/vector-api/products/901/coverage?time=1493751602099&apiKey=yourApiKey
Step 3: Get Features for a Single Tile
Required Parameters: productKey, time, lod, x, y, apiKey**=yourApiKey** **|| Optional Parameters:** declutter, tile-size https://api.weather.com/v2/vector-api/products/**<productKey>**/features.mvt?time=**<time>**&lod=**<lod>**&x=**<x>**&y=**<y>**&**apiKey=yourApiKey**
The [**/products/{productKey}/features**] request provides a set of features for a single tile, from a particular feature set within a particular product. Each feature contains a small set of key metadata properties, including its ID and valid time, which are required as input for any subsequent [**/products/{productKey}/feature-details**] request, as the **'feature-id'** and **'valid-time'** parameters.
https://api.weather.com/v2/vector-api/products/901/features?time=1492016701805&lod=10&x=175&y=409&tile-size=256&apiKey=yourApiKey https://api.weather.com/v2/vector-api/products/901/features.mvt?time=1492016701805&lod=10&x=175&y=409&tile-size=256&apiKey=yourApiKey
Step 4: Get All Properties for a Single Feature *(Optional)*
Required Parameters: productKey, feature-id, valid-time, apiKey**=yourApiKey** https://api.weather.com/v2/vector-api/products/**<productKey>**/feature-details?feature-id=**<featureId>**&valid-time=**<validTime>**&**apiKey=yourApiKey**
https://api.weather.com/v2/vector-api/products/901/feature-details?feature-id=3462696&valid-time=1493751602099&apiKey=yourApiKey

Summary of Endpoints

/vector-api/products/<productKey>/info
Retrieve details about a particular product, including time labels for the groups of feature sets that are currently available.

/vector-api/products/<productKey>/coverage
*Retrieve information on which tiles contain features, for a particular product and within a particular feature set, which is indicated by the **time** label that represents a discrete time value or a continuous time range*
The response's bitset is a "global" byte array, where each set bit (1) indicates that a particular tile contains at least one feature, while an unset bit (0) indicates the tile has no features.

- The data set starts from the lowest level of detail (**lod**=0, when **tile-size** is the default value of 256), iterates through each successive **lod** level, and ends with the max **lod** value for the product.
- For each level of detail, the data is sequenced in rows, left to right (west to east), beginning with the top (northmost) row.
- Bits are listed 0 to 7 in each byte, and the array terminates at the last set bit, with trailing zeros as needed to complete the last byte.

/vector-api/products/<productKey>/features
/vector-api/products/<productKey>/features.mvt
*Retrieve geographic features for a single tile, from a particular product and within a particular feature set, which is indicated by the **time** label that represents a discrete time value or a continuous time range*
The response's feature set includes features that are present, in whole OR in part, within that tile. The client-side application can use each feature's unique **id** to remove duplicate features from adjacent tiles; when retrieving a feature set for a continuous time range, the client can also use this identifier to choose between features that have the same **id** but different **validTime** values.

/vector-api/products/<productKey>/feature-details
Retrieve additional metadata details for one particular geographic feature
The response type depends on the specific endpoint that is called, as documented below.

REQUIRED: **All API responses must be compressed.** The .mvt file formats are compressed by definition; API requests for JSON objects, GeoJSON objects, and arrays of primitive data types **must** include the following HTTP header: "`Accept-Encoding: gzip`".		
Endpoint	Response	MIME Content-Type
/info	JSON	`application/json (gzipped)`
/coverage	bitset	`application/octet-stream (gzipped)`
/features	GeoJSON	`application/json (gzipped)`
/features.mvt	Mapbox Vector Tile	`vnd.mapbox-vector-tile`
/feature-details	JSON	`application/json (gzipped)`

Query Parameters: Elements & Definitions

Parameter Name	Description	Is Required	Sample
apiKey	Client's API Key	**Required**	apiKey=**yourApiKey**
/vector-api/products/<productKey>/info - Parameters Unique to Calls to Get Product Info			
Parameter Name	Description	Is Required	Sample
meta	Boolean: request to display metadata **Default Value:** false The metadata provides the geographic coverage and the level-of-detail, which conveys information about the valid ranges of **lod**, **x**, and **y** for a particular product. For each valid **lod** value, a geographically limited product may further restrict the valid ranges for **x** and **y** to include only those tiles that cover the product's region.	Optional	meta=true
max-times	Integer: maximum number of time-based labels to retrieve, with each label designating an available feature set When this parameter is positive, the system will return at most this number of times or ranges; when the parameter is negative, all times or ranges are returned; when the parameter is null or omitted, the system defaults to the product's configured, standard	Optional	max-times=12

	number of times or ranges Every product's features are grouped using one of two distinct schemes, either discrete time values or continuous time ranges.		
/vector-api/products/<productKey>/coverage - Parameters Unique to Calls to Get Coverage Info			
Parameter Name	Description	Is Required	Sample
time	Label for the time-based grouping of a particular feature set for the product Current labels are found in the response to calls to get product info, and a product's features are grouped using one of three distinct schemes. ● For a discrete time value, the format is the value's timestamp in milliseconds, in Epoch time. ● For a continuous time range, the format is **startTime-endTime:iterationNumber**, where each time is in milliseconds, in epoch time, and the range is followed by a colon and a zero-based iteration number. ● For a forecast product where the forecast is defined by a discrete model run time and a discrete forecast time, the format is **modelRunTime/forecastTime**, where each time is in milliseconds, in epoch time.	**Required**	time=1470117600000 time=1470117600000-1470139200000:0 time=1470117600000-1470139200000:1 time=1542024000000/1542067200000
/vector-api/products/<productKey>/features - Parameters Unique to Calls to Get Features for a Single Tile			
Parameter Name	Description	Is Required	Sample
time	Label for the time-based grouping of a particular feature set for the product Current labels are found in the response to calls to get product info, and every product's features are grouped using one of two distinct schemes. ● For a discrete time value, the format is the value's timestamp in milliseconds, in Epoch time. ● For a continuous time range, the format is **startTime-endTime:iterationNumber**, where each time is in milliseconds, in epoch time, and the range is followed by a colon and a zero-based iteration number. ● For a forecast product where the forecast is defined by a discrete model run time and a discrete forecast time, the format is **modelRunTime/forecastTime**, where each time is in milliseconds, in epoch time.	**Required**	time=1470117600000 time=1470117600000-1470139200000:0 time=1470117600000-1470139200000:1 time=1542024000000/1542067200000
lod	Integer: Level of Detail (LOD), also known as the zoom level The metadata found by calling **/vector-api/products/<productKey>/info?meta=true** provides geographic details dictating the valid ranges of **lod**, **x**, and **y** for a particular product. When **tile-size** is the default value of 256, 0 is the lowest **lod** value, indicating a global image tile in the Web Mercator projection. With the **tile-size** value of 512, 1 is the lowest **lod** value, again indicating a global tile. In either case, each subsequent **lod** value doubles the following: ● Zoom (level of detail) ● Range of possible **x** values ● Range of possible **y** values	**Required**	lod=4

x	Integer: horizontal position of the data tile at a particular level of detail Values start with 0 and increase left to right (west to east), bounded by approximately 180 degrees longitude. When tile-size is the default value of 256, the range is 0 to (2^z - 1) where z is the **lod** value. With the tile-size value of 512, the range is from 0 to [$2^{(z-1)}$ - 1].	**Required**	x=4
y	Integer: vertical position of the data tile at a particular level of detail Values start with 0 and increase top to bottom (north to south), bounded by approximately 85.051129 degrees North and South latitude. When tile-size is the default value of 256, the range is 0 to (2^z - 1) where z is the **lod** value. With the tile-size value of 512, the range is from 0 to [$2^{(z-1)}$ - 1].	**Required**	y=6
declutter	Boolean: request to perform point-decluttering and polygon-simplification **Default Value:** true When the parameter is set to false, the response is constructed from native data **when it is available**. Regardless of the value of this parameter, the system performs feature filtering, which determines whether to include features at a given **lod** level based on the properties of those features, such as a city's population size	Optional	declutter=false
tile-size	Offset for the **lod** parameter, configurable for legacy clients **Default Value:** 256 **Accepted Values:** 256 or 512 When **tile-size** is set to 512, the system decrements the **lod** parameter by 1. For example, the following (**x**, **y**, **lod**, **tile-size**) combinations correspond to the same tile, which encompasses the entire globe: (0, 0, 0, 256) (0, 0, 1, 512)	Optional	tile-size=512
/vector-api/products/<productKey>/feature-details - Parameters Unique to Calls to Get Additional Properties for a Single Feature			
Parameter Name	Description	Is Required	Sample
feature-id	Unique feature ID The value of this parameter is derived from the **id** field in the JSON object describing the feature, returned by tile-specific calls to **/vector-api/products/<productKey>/features**. Each whitespace character is replaced with %20.	**Required**	feature-id=ak13817038 feature-id=Dinagat%20Islands

	Combined, the **feature-id** and **valid-time** parameters uniquely identify a feature.		
valid-time	Unique valid time for the feature, generally the time of the event associated with a particular feature For most products, the value of this parameter is found in the **validTime** field in the JSON object describing the feature, returned by tile-specific calls to **/vector-api/products/<productKey>/features**. The field's value is a timestamp in milliseconds, in Epoch time. **NOTE** that, for certain forecast products, this query parameter must contain both the model runtime and the forecast time, in the format **modelRunTime/forecastTime**, where each timestamp is in milliseconds, in Epoch time. Both timestamps are found in the JSON object describing the feature; the model runtime will be in the **validTime** field, and the forecast time will be in a separate, product-specific custom field. In combination, the **feature-id** and **valid-time** parameters always uniquely identify a feature.	**Required**	valid-time=1470117600000 valid-time= 1542024000000/1542067200000

Responses: Elements & Definitions

Note: The table below does not necessarily represent the sort order of the API response.

Field Name	Description	Type	Range	Sample
/vector-api/products/<productKey>/info - Get Product Info				
detailsUrl	URL template for requesting details about a single geographic feature	string		http://api.weather.com/v2/vector-api/products/{productKey}/features/{featureKey}?&validtime={validTime}&apiKey={apiKey})
featuresUrl	URL template for requesting additional geographic features a for single tile, from a particular product and within a particular feature set	string		http://api.weather.com/v2/vector-api/products/{productKey}/features?time={time}&x={x}&y={y}&lod={lod}&apiKey={apiKey})
subdomains	URL subdomains that are available for domain sharding, used to increase the number of concurrent requests that a client may make To create valid URL endpoints, the client should replace the entire {subdomain} reference in the **featuresUrl** and **detailsUrl** properties with any of the values in this array.	[string]		["api", "api0", "api1", "api2", "api3"]
products	Object describing the attributes of one or more products, including, most notably, current labels for feature sets	object		
products.[productKey]	Object containing the attributes of a single product, including current labels for feature sets and optional metadata **NOTE** that this object's label is the actual product key for which productKey is ONLY a	object		

	placeholder, for example 601			
time	Array of currently available labels for a product's feature sets, which are grouped either by discrete time values, continuous time ranges, or the combination of a discrete model run time and a discrete forecast time A discrete time value is a single timestamp. A continuous time range has the format of **startTime-endTime:iterationNumber**, with a zero-based iteration number. For certain forecast product, where the forecast is defined by a discrete model run time and a discrete forecast time, the format is **modelRunTime/forecastTime**.	[string]	strings with timestamps in milliseconds, in Epoch time	*Discrete example:* 1470117600000 *Continuous range examples:* 1470117600000-1470139200000:0 1470117600000-1470139200000:1 *Forecast product example:* 1542024000000/1542067200000
products.[productKey].meta	Object containing metadata attributes for a product, including the geographic coverage and the level-of-detail pyramid This object is provided when the URL query string parameter **meta** is set to true.	object		
args	Configured values that are specific to a particular product definition, provided for the client's information	object		
coverage	Collection of bounding boxes describing the supported geographic coverage of the feature data Each bounding box object has four fields, describing its northern (**n**), eastern (**e**), southern (**s**), and western (**w**) borders. The value of each is field is in degrees of latitude or longitude.	[object]		{"w": -120.57, "e": -83.33, "n": 66.55, "s": -12.5}
defaultValidity	Configured amount of time in milliseconds, for the default validity for all features provided by this product This field’s value determines the validity for a feature in the absence of a specific value found in the field referenced by that feature's **validityProperty**. The **validity** of a feature indicates how long that feature may be acceptably shown in an animation or other visualization. This single property communicates both the 'forward validity' and the 'backward validity' of the feature, meaning that a feature's range of validity extends from (**validTime** - **validity**) to (**validTime** + **validity**). Each feature’s single valid time is used to assign that feature to a single unique group, even if its range of validity spans more than one group.	string	time in milliseconds	3898
direction	Direction in which the sequence of coordinate values are ordered This information can be useful for determining if a point has crossed the International Date Line (IDL). If this property is unavailable, this attribute will be omitted.	string	clockwise or counterclockwise	clockwise
properties	All known properties available with the product’s features	[string]		

pyramid	Array listing the product's key level-of-detail values The array begins with the minimum zoom level and ends with the maximum, non-decluttered zoom level. Each number in between indicates a level of detail where the decluttering rules change, resulting in a different set of features.	[integer]	Natural numbers, where 0 is the lowest possible zoom level	[3, 5, 7]
validityProperty	Field name found in ingest metadata for the product This field is used to store each feature's individual validity, a single value representing both the forward validity and the backward validity. The contents of this field supersede any **defaultValidity** set for the overall product.	string		val
/vector-api/products/<productKey>/features - Get Features for a Single Tile ***A successful response is a gzipped GeoJSON object where only the additional, non-standard attributes are described here; standard GeoJSON attributes are defined at http://geojson.org/ .***				
Field Name	Description	Type	Range	Sample
features	Array of objects, with each object describing a single feature **NOTE** that the fields for each feature object are described in the next section.	[object]		
id	Required feature identifier When used as the **feature-id** parameter in the URL for subsequent API calls, each whitespace character in this string should be replaced with %20.	string		Dinagat Islands ti17Kts b0hq0bz
properties	Object containing feature properties, including properties which are unique to the specific product and/or feature Calls to **/vector-api/products/<productKey>/features** return a small set of key properties for each feature, including the properties listed below. Additional properties can be retrieved by calling **/vector-api/products/<productKey>/feature-details**. **NOTE:** For details on these properties, consult the product's data dictionary document.	object		
intensity	Total number of point features represented by this single feature This value is included in a response that has undergone point decluttering, and the count includes this particular point feature along with the omitted point features.	integer		
validity	Duration for which a feature is valid; omitted if the value cannot be determined The field represents both the forward validity and the backward validity, centered at the **validTime**. The field's value is determined by the content of the ingest field referenced in the product's **validityProperty**, if it is available, or else by its product-wide **defaultValidity**.	integer	time in milliseconds	60000
validTime	The feature's required valid time	integer	epoch, in milliseconds	1470139200000

/vector-api/products/<productKey>/feature-details - Get Additional Properties for a Single Feature				
Field Name	Description	Type	Range	Sample
[unnamed object]	Object containing feature properties, including properties which are unique to the specific product and/or feature; the response may include properties omitted from the response to **/vector-api/products/<productKey>/features**. Because the feature ID and valid-time values are used in the API call producing this response, these properties are extraneous and may be omitted. **NOTE:** For details on these properties, consult the product's data dictionary document.	object		

Appendix: Valid Times, Validity, and Feature Sets

A feature's **valid time** is generally the time of the event associated with that feature.

The **validity** of a feature indicates how long that feature may be acceptably shown in an animation or other visualization. This single property communicates both the 'forward validity' and the 'backward validity' of the feature, meaning that a feature's range of validity extends from (**validTime** - **validity**) to (**validTime** + **validity**).

Each feature has a single valid time, and that valid time is used to assign the feature to a unique group, even if its range of validity spans more than one group. Every product's features are grouped using one of three distinct schemes.

- A series of *discrete time values*, such as **1470117600000**, which represents Tue, 02 Aug 2016 06:00:00 GMT in epoch format, in milliseconds. Each feature grouped with this value will have the same, matching valid time.
- A series of *continuous time ranges*, such as **1470117600000-1470139200000:0**, which represents the duration from Tue, 02 Aug 2016 06:00:00 GMT to Tue, 02 Aug 2016 12:00:00 GMT in epoch format, in milliseconds; the 0 following the colon (:) indicates that the contents of this 'bucket' have not been updated. Each feature grouped with this example time range will have a valid time somewhere within the six hours indicated.
- A series of *forecast times*, where each forecast is defined by a discrete model run time and a discrete forecast time, such as **1542024000000/1542067200000**, for a model run time of Mon, 12 Nov 2018, 00:00:00 GMT and a forecast time of Tue, 13 Nov 2018, 00:00:00 GMT, both in epoch format, in milliseconds. **NOTE** that only the *model run time* will be present in the feature's **validTime** field.

The feature set for a continuous time range may be updated multiple times, such as when the current time moves through that particular range. Updates to this 'bucket' are reflected with a (zero-based) iteration number, which is used in a sequence of labels for a single, evolving feature set.

The following example shows the first three labels for a single continuous time range, for which all labels have the format of **startTime-endTime:iterationNumber**.

- **1470117600000-1470139200000:0** is the initial label for the feature set; this indicates the first iteration of the feature set, prior to any updates.
- **1470117600000-1470139200000:1** indicates that one update has occurred.
- **1470117600000-1470139200000:2** indicates that two updates have occurred, etc.

Only the most recent label for each time range will be returned by calls to **/vector-api/products/<productKey>/info**, but each previous label can still be used to retrieve feature sets.

When the first call is made using a particular label, the current contents of the 'bucket' are cached and permanently associated with that label. If a newer label has been created before this first API call is received, the cache's contents will actually be more current than the iteration number would otherwise indicate.

It is **STRONGLY** recommended that the client always use the most recent label for a continuous time range, to ensure that the response includes the most current feature set possible.

Appendix: Tiles and the Tile-Size Property

The **tile** concept is used to allow clients to retrieve features in a specific geographic region at a particular level of detail (**lod**), also known as a zoom level.

- All tiles are defined from the Web Mercator projection.
- When the **tile-size** property has the default value of 256, the lowest **lod** level is 0; when **tile-size** is 512, the lowest **lod** level is 1.
    - At this lowest **lod** level, there is a single image tile encompassing the entire globe, and this single tile has an (**x**,**y**) ordered pair of (0,0).
- Each subsequent **lod** level doubles the zoom and doubles the ranges of possible **x** and **y** values.
- Values for x increase from west to east, and values for y increase from north to south.

For any nonnegative **lod** value z, when **tile-size** has the default value of 256, the ranges of **x** and **y** values are defined by the following inequality statements.

$$0 <= x <= 2^z - 1$$
$$0 <= y <= 2^z - 1$$

When **tile-size** is 512, **x** and **y** range from 0 to $2^{(z-1)} - 1$.

For example, at the **lod** value of 5, when **tile-size** is the default 256, there are 32 ($=2^5$) possible **x** values and 32 possible **y** values, with the first, northwesternmost tile at (0,0) and the last, southeasternmost tile at (31, 31).

When **tile-size** is 512, there are 16 ($=2^4$) possible x and values and 16 possible y values, and the tiles range from (0,0) to (15, 15).

Appendix: Response Optimization

By default, the Feature Data Service API carries out the following processes in calls to **/vector-api/products/<productKey>/features**, in order to optimize the response size as well as ensure computational efficiency and provide appropriate data for the tile's zoom level.

- **Point decluttering** limits the number of point features returned within a certain area; a single point feature is arbitrarily selected to represent all the points in that area, and this point feature contains a property called **intensity** which conveys the total number of points that it represents, including itself.
- **Polygon simplification** reduces the number of vertices in all polygon features, decreasing the complexity of each feature by approximating its true coverage.
- **Feature filtering** removes features based on their attributes. For instance, cities with a small population may be included at a high **lod** level but omitted from lower **lod** levels.

(**NOTE** that point decluttering does NOT entail clustering, where the properties of the set of points are aggregated and reflected in the representative point. Other than **intensity**, the values of the representative point's properties ONLY reflect that single point.)

When the native data is available, point decluttering and polygon simplification can be circumvented by setting the optional parameter **declutter** to false. The subsequent response is constructed from this native data but is still optimized with feature filtering.

Appendix: Coverage Response Explained

The following table summarizes the tiles referenced in the first three bytes of data returned from **/vector-api/products/<productKey>/coverage**. Bits are zero-indexed, here the first byte is labeled as Byte 0, and the first row for each **lod** is labeled as Row 0 -- and the first **lod** value is 0 when **tile-size** is the default value of 256.

Byte	0								1								2							
Bit Index	**0**	**1**	2	3	4	**5**	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	**21**	22	23
lod	0	1				2																3		
Row	0	0		1		0				1				2				3				0		
Tile (**x**,**y**)	**(0,0)**	**(0,0)**	(1,0)	(0,1)	(1,1)	**(0,0)**	(1,0)	(2,0)	(3,0)	(0,1)	(1,1)	(2,1)	(3,1)	(0,2)	(1,2)	(2,2)	(3,2)	(0,3)	(1,3)	(2,3)	(3,3)	**(0,0)**	(1,0)	(2,0)

The following formula produces the zero-indexed bit index **i** for a particular tile, where **z** is the tile's **lod** value (when **tile-size** is 256), **x** is the tile's horizontal position, and **y** is the tile's vertical position.

$$i = (4^z - 1)/3 + 2^z(\mathbf{y}) + \mathbf{x}$$

This table provides the first bit index for each **lod** value, up to an **lod** value of 6. The index is found by using the formula above, for tiles where **x** = 0 and **y** = 0.

Level of detail (**lod**), when **tile-size** is 256	(4^z - 1) / 3, where **z** is the **lod** value	Bit index for the first tile at this value (zero-indexed)
0	$(4^0 - 1) / 3 = (1 - 1) / 3 = 0/3 = 0$	0 (See the table above)
1	$(4^1 - 1) / 3 = (4 - 1) / 3 = 3/3 = 1$	1 (See the table above)
2	$(4^2 - 1) / 3 = (16 - 1) / 3 = 15/3 = 5$	5 (See the table above)
3	$(4^3 - 1) / 3 = (64 - 1) / 3 = 63/3 = 21$	21 (See the table above)
4	$(4^4 - 1) / 3 = (256 - 1) / 3 = 255/3 = 85$	85
5	$(4^5 - 1) / 3 = (1024 - 1) / 3 = 1023/3 = 341$	341
6	$(4^6 - 1) / 3 = (4096 - 1) / 3 = 4095/3 = 1365$	1365

Document Revision History

Revision	Date	Notes
1.0	Dec 7, 2018	Initial versioned document; addition of Document Revision History